IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 14 September 2024, accepted 11 October 2024, date of publication 17 October 2024, date of current version 30 October 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3482389

RESEARCH ARTICLE

Robust Vulnerability Detection in Solidity-Based Ethereum Smart Contracts Using Fine-Tuned Transformer Encoder Models

THI-THU-HUONG LE[1,2], JAEHYUN KIM[3], SANGMYEONG LEE[3], AND HOWON KIM[3], (Member, IEEE)

[1]Blockchain Platform Research Center, Pusan National University, Busan 609735, South Korea
[2]IoT Research Center, Pusan National University, Busan 609735, South Korea
[3]School of Computer Science and Engineering, Pusan National University, Busan 609735, South Korea

Corresponding authors: Thi-Thu-Huong Le (lehuong7885@gmail.com) and Howon Kim (howonkim@pusan.ac.kr)

This research was supported by the MSIT (Ministry of Science and ICT), Korea, under the ITRC (Information Technology Research Center) support program (IITP-2024-RS-2020-II201797) supervised by the IITP (Institute for Information & Communications Technology Planning & Evaluation); and in part by the MSIT (Ministry of Science and ICT), Korea, under the Convergence security core talent training business (Pusan National University) support program (No.RS-2022-II221201) supervised by the IITP (Institute for Information & Communications Technology Planning & Evaluation).

ABSTRACT The rapid expansion of blockchain technology, particularly Ethereum, has driven widespread adoption of smart contracts. However, the security of these contracts remains a critical concern due to the increasing frequency and complexity of vulnerabilities. This paper presents a comprehensive approach to detecting vulnerabilities in Ethereum smart contracts using pre-trained Large Language Models (LLMs). We apply transformer-based LLMs, leveraging their ability to understand and analyze Solidity code to identify potential security flaws. Our methodology involves fine-tuning eight distinct pre-trained LLM models on curated datasets varying in types and distributions of vulnerabilities, including multi-class vulnerabilities. The datasets-SB Curate, Benmark Solidity Smart Contract, and ScrawID-were selected to ensure a thorough evaluation of model performance across different vulnerability types. We employed over-sampling techniques to address class imbalances, resulting in more reliable training outcomes. We extensively evaluate these models using precision, recall, accuracy, F1 score, and Receiver Operating Characteristics (ROC) curve metrics. Our results demonstrate that the transformer encoder architecture, with its multi-head attention and feed-forward mechanisms, effectively captures the nuances of smart contract vulnerabilities. The models show promising potential in enhancing the security and reliability of Ethereum smart contracts, offering a robust solution to challenges posed by software vulnerabilities in the blockchain ecosystem.

INDEX TERMS Ethereum smart contracts, large language models, multi-class imbalance, multi-class classification, smart contract vulnerability, solidity code.

I. INTRODUCTION

The security of Ethereum smart contracts, particularly those developed using Solidity, is essential for ensuring the integrity of decentralized applications. Identifying and mitigating vulnerabilities have become increasingly challenging as these contracts become more complex. Traditional vulnerability detection methods often encounter difficulties related to scalability and precision, underscoring the need for more advanced approaches.

Vulnerability detection in Ethereum smart contracts primarily relies on static and dynamic analysis tools. Static analysis tools, such as Mythril [1] and Slither [2], analyze contract code without execution to detect vulnerabilities through pattern recognition and predefined rules. Although effective for common vulnerabilities like reentrancy and

The associate editor coordinating the review of this manuscript and approving it for publication was Domenico Rosaci.

© 2024 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

integer overflows, these tools can generate false positives and may overlook context-specific vulnerabilities due to their rule-based nature. Conversely, dynamic analysis tools such as Echidna [3] and Manticore [4] execute smart contracts in a simulated environment, exploring various execution paths to uncover vulnerabilities. While dynamic analysis can identify complex issues that static tools may miss, it requires significant computational resources and is limited by the scope of test scenarios. Both approaches' distinct strengths and weaknesses suggest that integrating them with machine learning techniques could enhance vulnerability detection by fusing the advantages of static and dynamic analyses with advanced pattern recognition and classification capabilities [5], [6].

Recent advancements in machine learning and natural language processing (NLP) offer promising solutions to these challenges. Text-based Large Language Models (LLMs), especially those based on transformer architectures, have performed exceptionally in various NLP tasks, including text classification and anomaly detection. Applying these models to Solidity code presents a significant opportunity for improving vulnerability detection in smart contracts [7], [8].

A critical challenge in this domain is the management of multi-class data imbalances. Vulnerabilities in smart contracts can be categorized into numerous classes, and a single contract may present multiple vulnerabilities simultaneously. Effectively addressing these imbalances is vital for enhancing detection performance. Consequently, techniques for managing multi-class imbalance in machine learning models are essential for accurate vulnerability identification [10], [11], [12].

Recent research has investigated the potential of fine-tuning LLMs for code analysis and vulnerability detection [13]. Fine-tuning pre-trained LLMs, such as BERT, for code-related tasks has shown notable improvements in identifying code anomalies and vulnerabilities [14]. For instance, Thapa et al. [15] applied BERT-based models to software vulnerability detection, demonstrating enhanced classification accuracy and robustness. Similarly, GPT-3 has been employed for vulnerability detection in smart contracts, yielding significant advancements in detection performance [16], [17], [18].

Further studies have explored the application of LLMs in code analysis, highlighting transformer models for code vulnerability detection. These investigations emphasize their efficacy in identifying code flaws [15], [19]. Moreover, they underscore the utility of transformer-based models in enhancing smart contract security [20] and provide insights into improving detection accuracy for complex code scenarios through deep learning methods for multi-class vulnerability detection [21].

This paper builds upon these advancements by fine-tuning text-based LLMs specifically for vulnerability detection within Solidity code. Our approach confronts the challenges of managing multi-class imbalances using innovative techniques such as Random Sampling for class imbalance and tailored strategies for multi-class classification. We evaluate our methodology by comparing eight state-of-the-art models across three publicly available Solidity datasets.

The primary contributions of this paper include:

- LLM-Based Vulnerability Detection: We utilize transformer-based LLMs to analyze Solidity code, leveraging their advanced pattern recognition and contextual understanding capabilities to identify various vulnerabilities.
- Addressing Multi-Class Imbalances: Our approach effectively tackles significant challenges associated with data imbalances in multi-class classification through innovative preprocessing techniques.
- Comprehensive Evaluation: Using eight pre-trained LLMs, we conduct extensive experiments on three distinct datasets–SB Curate, Benmark Solidity Smart Contract, and ScrawID. Our evaluation employs rigorous metrics, including precision, recall, F1 score, and ROC-AUC, to assess model performance.
- Insights for Smart Contract Security: The findings offer valuable insights for enhancing smart contract security by applying advanced machine learning techniques, providing a robust framework for future research in this domain.

The structure of this paper is as follows: In Section II, Related Work, we review the literature on vulnerability detection in smart contracts, focusing on the application of machine learning and LLMs in this field. Section III, Methodology, details the approach employed in our study, including the selection of datasets, model architectures, and techniques utilized to address data imbalance and multi-class classification challenges. In Section IV, Experimental Results, we present the findings of our experiments, evaluating the performance of the eight pre-trained LLM models across various metrics. Section V, Discussion, provides an in-depth analysis of the results, highlighting the implications of our findings for improving smart contract security. Finally, Section VI, Conclusion, summarizes the key contributions of our work and outlines potential directions for future research.

II. RELATED WORK

This section reviews the relevant literature and prior research that form our study's basis. We focus on works related to vulnerability detection in smart contracts, the application of machine learning models to software security, LLMs, and datasets used for training and evaluation.

A. MACHINE LEARNING VULNERABILITY DETECTION IN SMART CONTRACTS

Recent advancements in machine learning have shown promising results in various domains, including software security. Reference [22] explored using Convolutional Neural Networks (CNNs) for vulnerability detection in code. Their approach demonstrated the ability to classify code snippets based on known vulnerability types, contributing to the development of automated detection systems.

Smart contract vulnerabilities have been a significant concern in blockchain security. Early work in this area often relied on static analysis tools to detect common vulnerabilities. For example, [23] proposed a static analysis approach for identifying vulnerabilities in Ethereum smart contracts. Their method used symbolic execution to uncover potential issues such as reentrancy and integer overflow.

Building upon static analysis, researchers began exploring machine-learning techniques to enhance vulnerability detection. References [24] and [25] introduced deep learning-based approaches to identify vulnerabilities in smart contracts. They leveraged neural networks to learn from code patterns and detected vulnerabilities with improved accuracy compared to traditional methods.

Furthermore, [26] investigated the application of Transformer models to code analysis tasks. They found that with their attention mechanisms, transformer-based models could capture complex code relationships and improve detection performance. This research supports using advanced neural architectures for vulnerability detection in smart contracts.

Recent work by [27] proposed a novel graph neural network approach for smart contract vulnerability detection. Their method utilizes a hierarchical graph structure to capture syntactic and semantic information from smart contract code, demonstrating superior performance compared to previous methods.

Lastly, [28] introduced a vulnerability detection framework that combines code semantics and control flow information for vulnerability detection. Their approach showed significant improvements in detecting complex vulnerabilities that require understanding code structure and execution flow.

B. LLM VULNERABILITY DETECTION IN SMART CONTRACTS APPROACHES

LLMs have transformed the landscape of natural language processing (NLP) and are increasingly utilized in various domains, including software engineering and vulnerability detection. Their ability to comprehend and generate human-like text has been extended to understanding programming languages, such as Solidity, used in smart contracts. This subsection explores the methodologies and approaches of applying LLMs to detect vulnerabilities in smart contracts.

Pre-trained transformer models, such as BERT [29], GPT [30], and their variants, have demonstrated significant performance improvements across various NLP tasks. These models are pre-trained on extensive text corpora and fine-tuned for specific tasks. LLMs are fine-tuned on domain-specific datasets for smart contract vulnerability detection to enhance their understanding of programming languages and associated vulnerabilities.

BERT-based models, including BERT itself, DistilBERT [31], and RoBERTa [32], use bidirectional context to capture relationships within the code. This bidirectional approach allows the models to grasp the semantics of programming constructs and identify potential vulnerabilities effectively. For instance, DistilBERT and RoBERTa have been successfully applied to analyze Solidity code, leveraging their contextual understanding to detect vulnerabilities [33].

Generative Pre-trained Transformers (GPT), such as GPT-2 [34] and GPT-3 [35], provide another approach for code analysis. Unlike BERT, GPT models are autoregressive and generate text based on prior inputs. Fine-tuning these models on code-related tasks can facilitate code completion and vulnerability detection, making them valuable for identifying security flaws in smart contracts [18], [36].

Fine-tuning is a critical step where pre-trained models are adapted to specific tasks by training them on smaller, task-specific datasets. Vulnerability detection in smart contracts involves training LLMs on annotated datasets with vulnerable and secure code examples. Datasets such as SB Curate [37], Benmark Solidity [38], and ScrawID [39] are used for this purpose. Proper dataset preparation, including tokenization and encoding of vulnerabilities, is essential for effective model fine-tuning.

Recent advancements have shown promising results in applying LLMs to smart contract vulnerability detection. For example, [40] proposed SmartLLM, a large language model specifically designed for smart contract analysis, demonstrating superior performance in detecting complex vulnerabilities compared to traditional methods. Additionally, [41] introduced SCVulBERT, a BERT-based model pre-trained on a large corpus of smart contracts, which achieved state-of-the-art results in various smart contract analysis tasks, including vulnerability detection.

Despite their effectiveness, LLMs face several challenges in vulnerability detection, including handling imbalanced datasets, understanding complex code patterns, and generalizing across diverse codebases. Reference [42] addressed some of these challenges by integrating a multitask sequence-to-sequence LLM with program control flow graphs encoded as a graph neural network to achieve sequence-to-classification vulnerability detection.

In summary, LLMs represent a powerful tool for detecting vulnerabilities in smart contracts by leveraging pre-trained knowledge and fine-tuning capabilities. Continued advancements in model architectures, training methodologies, and integration strategies will further enhance their effectiveness in ensuring secure and reliable smart contracts.

C. DATASETS FOR TRAINING AND EVALUATION

The performance of machine learning models is significantly influenced by the quality and size of the datasets used for training. As highlighted by [43], large-scale datasets are crucial for developing robust vulnerability detection models, as they can greatly enhance model performance and generalization capabilities.

Several datasets have gained prominence in the realm of smart contract vulnerability research. For this study, we selected three notable datasets, each offering distinct attributes that contribute to a comprehensive evaluation of vulnerability detection models:

- SB Curate: The SB Curate dataset [37] is renowned for its curated collection of Ethereum smart contract IDs with labeled vulnerabilities. It provides a structured dataset with clear categorization of vulnerabilities, which is essential for training models to recognize specific flaws. The structured labeling allows for analyzing model performance across various vulnerabilities and ensures that the models can generalize well to different vulnerability classes.
- Benchmark Solidity Smart Contract: The Benchmark dataset [38] comprises a vast array of Solidity code samples. Its extensive coverage of smart contract code is particularly valuable for training models on diverse code patterns and structures. The large sample size enhances the model's robustness and ability to detect vulnerabilities across various coding styles and practices. Additionally, the variety of code samples enables the assessment of model performance on different contract types and implementations.
- ScrawID: The ScrawID dataset [39] offers a diverse set of smart contracts with multiple vulnerabilities. This diversity makes it a valuable resource for evaluating detection models, including single-label vulnerability types. The presence of multiple vulnerability types allows for a comprehensive assessment of the model's ability to detect various kinds of vulnerabilities simultaneously. The diversity also aids in understanding how well models can handle complex scenarios where multiple vulnerabilities might occur within a single contract.

The selection of these three datasets ensures a thorough evaluation of vulnerability detection models. SB Curate provides detailed vulnerability labeling, Benmark offers extensive code samples for training, and ScrawID introduces complexity with multiple vulnerability types. Together, these datasets encompass a wide spectrum of smart contract vulnerabilities and coding practices, enabling a robust assessment of model performance and generalization.

D. GAPS AND OPPORTUNITIES

Despite significant advancements in smart contract vulnerability detection, several gaps persist in the current research landscape:

- Imbalanced Datasets: Existing methods often encounter difficulties when dealing with imbalanced datasets, which can result in biased predictions. Addressing this issue is crucial for improving model performance. Techniques such as oversampling, undersampling, or synthetic data generation can mitigate the effects of imbalance and enhance the reliability of predictions.
- Diversity Multi-Class Classification: There is considerable potential for further exploration of multi-class classification techniques in vulnerability detection. Current approaches predominantly focus on single-label classification. Expanding research in this area could improve the accuracy and utility of detection systems by allowing them to identify and categorize multiple vulnerabilities within a single smart contract.

In summary, while significant progress has been made in using machine learning for smart contract vulnerability detection, ongoing research is essential to address these challenges and explore new methodologies. By focusing on these gaps, the field can continue to evolve and improve the effectiveness of vulnerability detection systems.

III. METHODOLOGY

This section details our methodology, as illustrated in Figure 1. Our approach began with preprocessing and customizing the dataset from the originally published sources to align with our objectives. We converted Solidity code from text into tokenizer and tensor data for input into the transformer encoder, which allowed the Large Language Model (LLM) to understand and generate embedding tensors with appropriate padding positions. During the Supervised Fine-Tuning (SFT) phase, the text-based LLM, incorporating the transformer encoder component, processed the tensor data to identify vulnerabilities in Solidity code and make final classification decisions. This process ensured the model was well-equipped to detect and classify vulnerabilities accurately.

A. PREPROCESSING DATA

We applied our method to three distinct published datasets. The first dataset, referred to as DS1, is SB Curate [37]. The second dataset denoted as DS2, is the benchmark dataset of Solidity smart contracts [38]. The third dataset, labeled as DS3, is ScrawID [39]. Key information about these datasets is summarized in Table 1.

The diversity in vulnerability labels between datasets arises from differences in the methods and sources used for collecting and labeling the Solidity code. As noted in Table 1 of our paper, each dataset (DS1, DS2, and DS3) is derived from distinct Solidity code collections, which leads to variations in the labeling process and the types of vulnerabilities reported.

- DS1 and DS2: Although both datasets contain 10 classes, the specific labels differ due to different smart contract collections and attack types. We maintained the original label names to respect the dataset authors' work.
- DS3: This dataset features both single-label and multi-label classifications. Some Solidity smart contracts are labeled with a single vulnerability, while others have multiple vulnerabilities. For example, a contract might be labeled with `ARTHM` (Arithmetic) and `LE` (Locked Ether). We combined these labels (e.g., `ARTHM_LE`) to reflect the multi-label nature of DS3. This approach was necessary because DS3 uses various detection tools that identify multiple vulnerabilities within a single contract.

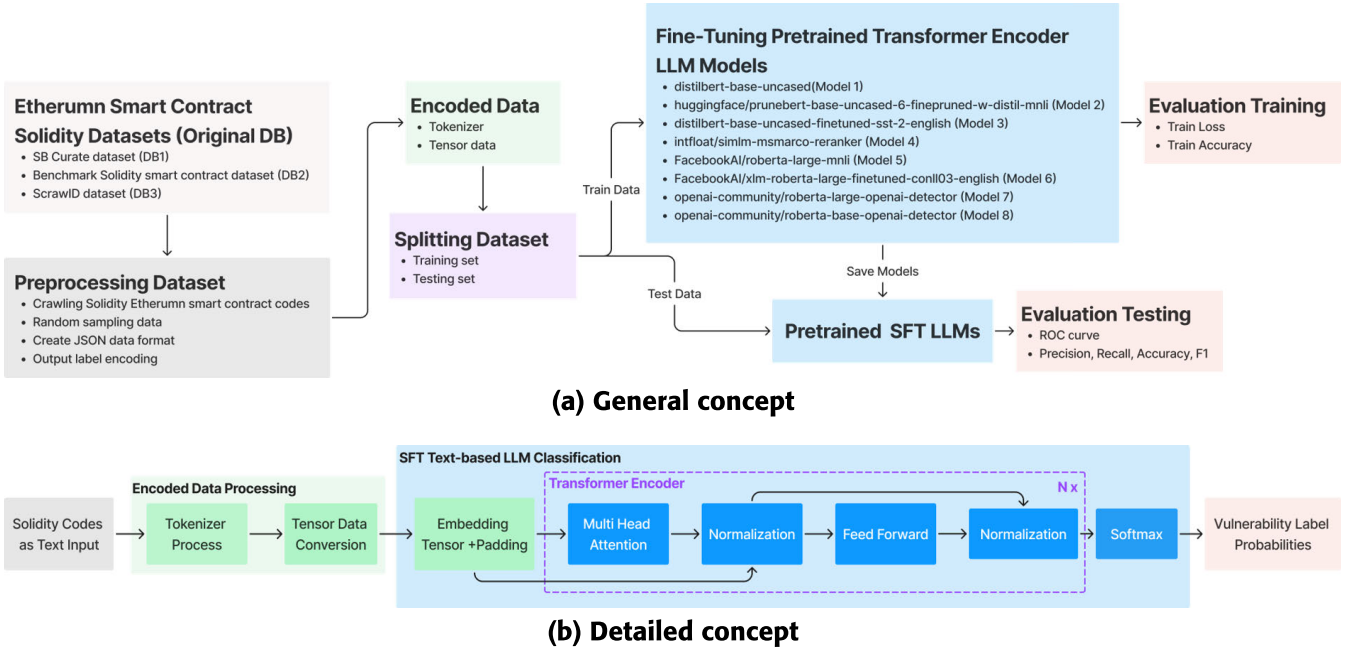


FIGURE 1. Fine-tuning of pretrained transformer encoder models for vulnerability detection in LLMs: (a) General proposed concept; (b) Detailed proposed concept.

TABLE 1. Datasets description.

Dataset	No. of Samples	No. of Output Labels	Vulnerability Labels
DS1	143	10	access_control, arithmetic, bad_randomness, denial_of_service, front_running, reentrancy, short_addresses, time_manipulation, unchecked_low_level_calls, other
DS2	4,364	10	DC1, FE1, IOU1, NC1, RE1, TD1, TO1, TOD1, UcC1, UpS1
DS3	5,523	36	ARTHM, ARTHM_LE, ARTHM_LE_RENT, ARTHM_LE_RENT_TimeO, ARTHM_LE_RENT_TX-Origin, ARTHM_LE_TimeM, ARTHM_LE_TimeM_TimeO, ARTHM_LE_TimeO, ARTHM_LE_TX-Origin, ARTHM_LE_UE, ARTHM_RENT, ARTHM_RENT_TimeM, ARTHM_RENT_TimeM_TimeO, ARTHM_RENT_TimeO, ARTHM_RENT_TX-Origin, ARTHM_TimeM, ARTHM_TimeM_TimeO, ARTHM_TimeO, ARTHM_TX-Origin, ARTHM_UE, LE, LE_RENT, LE_RENT_TimeO, LE_TimeM, LE_TimeM_TimeO, LE_TimeO, LE_UE, RENT, RENT_TimeM, RENT_TimeM_TimeO, RENT_TimeO, TimeM, TimeM_TimeO, TimeO, TX-Origin_TimeO, UE

It is important to note that DS1 and DS3 do not directly provide publicly available Solidity code. Therefore, we had to crawl the Ethereum official website[1] based on the smart contract IDs provided, whereas DS2 was readily available. Another challenge was the imbalanced distribution of vulnerability types across all three datasets. In multi-class classification problems, datasets often suffer from imbalanced class distributions, where some classes are significantly underrepresented relative to others. This imbalance can adversely affect model performance, leading to biased predictions. Random Over-Sampling (ROS) is designed to mitigate this issue by augmenting the number of instances in the minority classes.

Class imbalance is a prevalent issue in multi-class classification tasks, where certain classes are underrepresented compared to others. ROS technique offers a straightforward approach to address this imbalance by increasing the representation of underrepresented classes. This method involves replicating instances from minority classes to achieve a balanced dataset. Therefore provide a detailed description of the ROS method, including its mathematical formulation and procedural steps for multi-class scenarios.

Given a multi-class dataset D with K classes: C_1, $C_2, \ldots, C_K$, where: $|D_i|$ denotes the number of instances in class C_i (for $i = 1, 2, \ldots, K$), the procedure for ROS involves the following steps:

1) **Identify Imbalance**: Determine the number of instances in each class. Identify the classes with the least instances, which are considered the minority classes.
2) **Replication**: For each minority class C_i, replicate instances to balance the number of instances with the majority class (or the class with the highest number of instances). Specifically:
   - Let $|D_{\max}|$ be the maximum number of instances among all classes.

[1] https://etherscan.io/



FIGURE 2. Distribution sample data for imbalanced data and sampled data via each vulnerability types three datasets. (a) and (b) SBCurate (DS1); (c) and (d) Benchmark smart contract (DS2); (e) and (f) ScrawID (DS3).

- For each minority class C_i, if $|D_i| < |D_{\max}|$, generate additional instances to reach $|D_{\max}|$. This involves randomly duplicating instances from D_i.

3) **Mathematical Formulation**: Calculate the number of duplicates required for each class C_i. Let k_i be the number of duplicates needed for class C_i. The number of additional instances k_i can be computed as:

$$k_i = \frac{|D_{\max}| - |D_i|}{|D_i|} \tag{1}$$

where k_i represents the number of duplications needed to balance class C_i with $|D_{\max}|$. To achieve the desired balance, instances in C_i times are randomly duplicated k_i times.

The results of the balanced dataset obtained after applying the sampling method are shown in Figures 3b, 3d, and 3f.

We created three JSON datasets from the balanced data. The JSON format of these datasets includes input data, consisting of the Solidity code, and output data, corresponding to the vulnerability labels. These datasets are saved, and output labels are encoded with the LLM models in subsequent stages.

B. ENCODED AND SPLITTING DATA

Tokenization involves breaking down text into smaller units, or tokens, such as words, phrases, subwords, or characters. For Transformer models, effective tokenization is essential

for preprocessing text data, ensuring that the model can learn meaningful patterns from the input.

In the case of programming languages like Solidity, tokenization must accurately capture syntax, keywords, identifiers, operators, and structure. The goals of tokenization are to (1) Break down the code into meaningful units (tokens). (2) Preserve the syntax and structure. (3) Handle special tokens for keywords, identifiers, operators, etc.

We applied the WordPiece method [44] for tokenizing Solidity code, a subword tokenizer type. We selected WordPiece for our work due to its effectiveness in handling programming languages and its ability to manage rare and out-of-vocabulary tokens efficiently.

The data from JSON files can be read and accessed for the input and output features of LLM models. To prepare the data for training, we perform the following steps: (1) Data Encoding: Convert the raw data into a format suitable for the LLM models. This involves tokenizing the Solidity code and encoding the vulnerability labels to be compatible with the model's input requirements. (2) Data Splitting: Divide the dataset into training, validation, and test sets. This ensures the model is trained on one portion of the data, validated on another to tune hyperparameters, and tested on a separate set to evaluate its performance. These steps ensure the model can learn effectively and generalize well to unseen data.

1) Encoding Data: We first encode the data using a tokenizer. For a given input x (Solidity code) and output y (vulnerability labels), the tokenizer converts x into a sequence of tokens $T(x)$ and y into a set of encoded labels $E(y)$. Mathematically, this can be represented as:

$$T(x) = \text{Tokenizer}(x) \tag{2}$$

$$E(y) = \text{Encoder}(y) \tag{3}$$

2) Conversion to Tensor Data: After encoding, the tokenized data $T(x)$ and encoded labels $E(y)$ are converted into the tensor format. Let $\mathbf{X}$ denote the tensor representation of the input data and $\mathbf{Y}$ denote the tensor representation of the output labels. This conversion can be expressed as:

$$\mathbf{X} = \text{Tensor}(T(x)) \tag{4}$$

$$\mathbf{Y} = \text{Tensor}(E(y)) \tag{5}$$

3) Data Splitting: We split the dataset into training, validation, and test sets to train and evaluate our models. Let D denote the entire dataset. We split D into training D_{train}, validation D_{val}, and test D_{test} subsets, where:

$$D_{\text{train}} \cup D_{\text{val}} \cup D_{\text{test}} = D \tag{6}$$

and typically:

$$|D_{\text{train}}| : |D_{\text{val}}| : |D_{\text{test}}| = 70\% : 15\% : 15\% \tag{7}$$

The JSON data files are encoded using a tokenizer, converted to tensor format, and split into appropriate subsets for training, validation, and testing of the LLM models.

C. PRETRAINED TRANSFORMER ENCODER LLM MODELS

The data from JSON files are read and accessed to extract input and output features for the LLM models. The input data, consisting of Solidity code, is encoded by a tokenizer and converted into tensor format, including padding. This tensor data becomes the input for the transformer encoder LLM models.

We evaluated our approach using eight pre-trained LLMs, each selected for its unique characteristics and capabilities in natural language understanding and text classification.

1) `distilbert-base-uncased` [45] (Model 1): This model is a smaller, distilled version of BERT, known for its efficiency and effectiveness in various NLP tasks. Its reduced size, while retaining most of BERT's performance, makes it suitable for tasks requiring moderate computational resources.
2) `huggingface/prunebert-base-uncased-6-finepruned-w-distil-mnli` [46] (Model 2): This model is a pruned variant of BERT optimized for the MNLI (Multi-Genre Natural Language Inference) task. Its fine-tuned nature enhances its performance on inference tasks, which can be valuable for understanding complex patterns in vulnerability detection.
3) `distilbert-base-uncased-finetuned-sst-2-english` [47] (Model 3): This model, a fine-tuned version of DistilBERT on the SST-2 (Stanford Sentiment Treebank) dataset, excels in sentiment analysis. Its adaptation to sentiment-related tasks provides insights into its ability to discern nuanced aspects of the text.
4) `intfloat/simlm-msmarco-reranker` [48] (Model 4): This model is designed for re-ranking tasks based on the MS MARCO dataset. It excels in improving the relevance of ranked results, which can be useful for prioritizing vulnerabilities based on their criticality.
5) `FacebookAI/roberta-large-mnli` [49] (Model 5): A large version of RoBERTa fine-tuned on MNLI, this model offers robust performance on a range of natural language inference tasks, making it suitable for understanding complex relationships between vulnerabilities.
6) `FacebookAI/xlm-roberta-large-finetuned-conll03-english` [50] (Model 6): This model, fine-tuned on the CoNLL-03 dataset for named entity recognition (NER), is adept at identifying and categorizing entities within text, which is useful for extracting specific types of vulnerabilities from smart contracts.
7) `openai-community/roberta-large-openai-detector` [51] (Model 7): A specialized model designed to detect text generated by OpenAI's models, it is particularly effective in identifying synthetic text, which can be useful for filtering and validating code-related data.

8) `openai-community/roberta-base-openai-detector` [52] (Model 8): Similar to the previous model but with a base configuration, this model is optimized for detecting OpenAI-generated text, providing a more lightweight option for our vulnerability detection tasks.

Each model was fine-tuned on our specific dataset of smart contract vulnerabilities. The fine-tuning process involved updating the model weights using our labeled dataset, which consisted of input Solidity code and corresponding vulnerability labels. We performed fine-tuning by training each model for a set number of epochs with a learning rate adjusted to ensure convergence.

Training and Fine-Tuning: In general, in our fine-tuning approach, we utilized the following steps:

- Starting with Pre-Trained Weights: We initialized our model using weights from a pre-trained network. This strategy leverages features learned from a related task or dataset, providing a strong foundation for our model.
- Updating Weights: Contrary to freezing some layers, we updated the weights of all layers during the fine-tuning process. This approach ensures that the entire model can adapt and refine its weights to better align with the specific characteristics of our target dataset.

Specifically, the fine-tuning procedure for each model involves the following steps:

- Loading Pre-trained Model: We start with the pre-trained weights of the LLM.
- Input Preparation: Convert the Solidity code into tokenized and tensor data suitable for the model.
- Fine-tuning: Train the model on our dataset using a suitable optimizer and learning rate. We utilized a learning rate of 1×10^{-5} and the `AdamW` optimizer.
- Loss Calculation: The loss function used during fine-tuning is `CrossEntropyLoss`, which is appropriate for multi-class classification tasks. The loss is computed as:

$$\text{Loss} = -\sum_{i} Y_i \log(\hat{Y}_i)(1) \tag{8}$$

  where Y_i is the true label and $\hat{Y}_i$ is the predicted probability for class i.
- Model Evaluation: After fine-tuning, each model's performance is evaluated using metrics such as accuracy and F1-score.

The transformer encoder architecture used in our models includes several key components:

- Multi-Head Attention: This mechanism allows the model to focus on different parts of the input sequence simultaneously. The attention operation is repeated h times, where h denotes the number of attention heads.
- Normalization: Layer normalization stabilizes and accelerates training by normalizing the inputs to each layer. This process is defined as:

$$\text{LayerNorm}(x) = \frac{X - \mu}{\sigma + \epsilon} \cdot \gamma + \beta \tag{9}$$

  where X is the input tensor, μ and σ are the mean and variance, ϵ is a small constant, and γ and β are learnable parameters.
- Feed-Forward Layer: This consists of two linear transformations with a ReLU activation function in between:

$$\text{FFN}(x) = \text{ReLU}(XW_1 + b_1)W_2 + b_2 \tag{10}$$

  where W_1 and W_2 are weight matrices, and b_1 and b_2 are bias vectors.
- Normalization: Another normalization layer is applied after the feed-forward layer.

The encoder block, including multi-head attention, feed-forward layer, and normalization, is repeated N times, where N denotes the number of encoder layers. Finally, a softmax layer computes the probability distribution over vulnerability types:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \tag{11}$$

where z_i is the logit for the i-th vulnerability type, providing the probability distribution over all types.

This architecture allows us to leverage the strengths of each pre-trained LLM and evaluate their performance in detecting vulnerabilities in smart contracts.

Our transformer encoder models are configured with specific hyperparameters, summarized in Table 2. These hyperparameters are essential for effectively training the models, ensuring they are optimally tuned for detecting vulnerabilities in Solidity code.

- Input Feature: The input feature is set to 1, representing a single feature vector per token in the input sequence.
- Number of Label Outputs: For DS1, there are 10 label outputs. For DS2, there are 10 label outputs. For DS3, there are 36 label outputs.
- Padding: The padding size is set to 256. This ensures all input sequences are padded to a uniform length for batch processing.
- Truncation: Truncation can handle input sequences that exceed the maximum length of 256 tokens. This prevents the model from processing excessively long sequences.

TABLE 2. Hyperparameters setting.

Hyperparameters Name	Value	Note
Input feature	1	
Number of label outputs	10	DS1
	10	DS2
	36	DS3
Padding	256	
Truncation	True	
Batch size	8	
Learning rate	1e-5	
Optimizer	`AdamW`	
Loss function	`CrossEntropyLoss`	
Epochs	10	

- Batch Size: The batch size is set to 8, balancing the computational load and memory constraints during training.
- Learning Rate: The learning rate is set to 1×10^{-5}, which controls the rate at which the model updates its weights.
- Optimizer: The `AdamW` optimizer is used, an adaptation of the Adam optimizer with weight decay regularization.
- Loss Function: The `CrossEntropyLoss` function is employed to measure the performance of the classification model, particularly for multi-class classification tasks.
- Epochs: The model is trained for 10 epochs, allowing the model to iterate through the dataset multiple times for optimization.

D. METRIC EVALUATION ON LLM MODELS

Our models are developed using Python and popular machine-learning libraries. Specifically, we utilized Scikit-learn v1.5 for data preprocessing, encoding, and evaluation metrics. We employed PyTorch and the Hugging Face Transformers library for model development and fine-tuning. The experiments were conducted on an NVIDIA Tesla V100 DGX system with 32 GB of memory. The system specifications include NVIDIA-SMI 450.142.00, Driver Version 450.142.00, and CUDA Version 11.7.

We employ various metrics to evaluate our models' performance that provide insights into their predictive capabilities. During the training and validation phases, we utilize ROC curve analysis. We assess precision, recall, accuracy, and F1 score for the testing phase to evaluate the models' effectiveness comprehensively.

1) ROC CURVE AND AUC

The Receiver Operating Characteristic (ROC) curve represents a model's performance across different classification thresholds. It plots the True Positive Rate (TPR) against the False Positive Rate (FPR):

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{12}$$

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} \tag{13}$$

, where TP, FP, TN, and FN represent the counts of true positives, false positives, true negatives, and false negatives, respectively. The Area Under the Curve (AUC) measures the overall performance of the model:

$$\text{AUC} = \int_0^1 \text{TPR}(FPR)\, d(\text{FPR}) \tag{14}$$

2) PRECISION, RECALL, ACCURACY, AND F1 SCORE

For the evaluation of the testing models, we use the following metrics:

- Precision: Measures the proportion of true positive predictions out of all positive predictions made by the model:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{15}$$

- Recall: Measures the proportion of true positive predictions out of all actual positives in the dataset:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{16}$$

- Accuracy: Measures the proportion of correctly predicted instances (both true positives and true negatives) out of the total instances:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \tag{17}$$

- F1 Score: The harmonic mean of precision and recall, which provides a single metric that balances both concerns:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{18}$$

These metrics are calculated for each model to assess its performance in detecting vulnerabilities in smart contracts. The ROC curve and AUC provide an overall view of the model's ability to distinguish between classes, while precision, recall, accuracy, and F1 score offer detailed insights into the model's effectiveness in handling class imbalances and ensuring reliable predictions.

We comprehensively assess each model's performance and suitability for practical vulnerability detection applications by employing these evaluation metrics.

IV. EXPERIMENT RESULT

This section presents the experimental results, showcasing how our method effectively addresses existing challenges. We start by plotting training evaluation metrics, such as accuracy and loss, to demonstrate the performance of our proposed method. In addition, we present the results of the testing evaluation using ROC curve plots, which visualize the classification performance for each vulnerability label. Furthermore, we provide a detailed performance analysis of the best model in each dataset, including precision, recall, precision, and the F1 score evaluations.

A. TRAINING EVALUATION

We present the training evaluation results of our eight models under two distinct scenarios: imbalanced and balanced datasets, tested across three different datasets. The overall results indicate that balancing the datasets significantly improves model performance. Training on balanced datasets consistently leads to higher accuracy and lower loss than training on imbalanced datasets. The accuracy plots demonstrate that models trained on balanced datasets achieve higher accuracy across all datasets, reflecting improved generalization. In contrast, the loss curves reveal that models trained on balanced datasets converge more quickly and to a lower final loss, indicating better learning efficiency and stability.

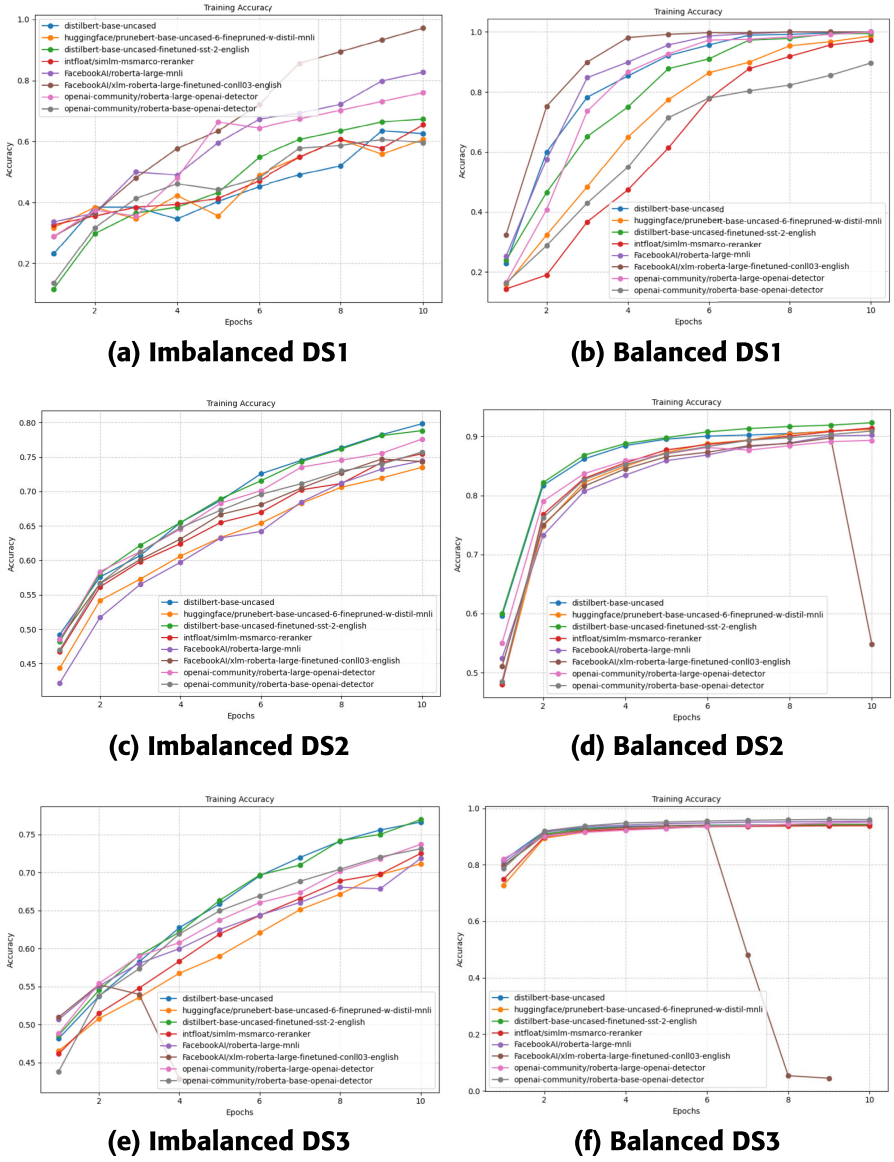


FIGURE 3. Training accuracy of eight models across three datasets. (a) On Imbalanced SB Curate Data; (b) On Balanced SB Curate Data; (c) On Imbalanced Benchmark Solidity Smart Contract Data; (d) On Balanced Benchmark Solidity Smart Contract Data; (e) On Imbalanced ScrawlD Data; (f) On Balanced ScrawlD Data.

1) TRAINING ACCURACY RESULTS

We evaluate the training accuracy of eight models across three imbalanced datasets (without sampling) and three balanced datasets (with sampling), as illustrated in Figure 3. For the imbalanced datasets, the training accuracy of most models falls below 80%, except for Model 5, which achieved over 90% accuracy on DS1 (see Figure 3a). However, Model 5's performance drops below 45% for DS3 (see Figure 3e).

In contrast, for the balanced datasets, Model 3 demonstrates the most consistent and competitive performance across all three datasets. Although Model 5 also achieves impressive performance, exceeding 99% accuracy on DS1 (see Figure 3b), it underperforms on two other balanced datasets, DS2 and DS3 (see Figures 3d and 3f).

2) TRAINING LOSS RESULTS

We assess the training loss across eight models for three imbalanced and three balanced datasets, as depicted in Figure 4. For the imbalanced datasets, most models maintain a training loss below 1.5. Notably, Model 6 achieves an exceptionally low loss of under 0.3 on DS1 (refer to Figure 4a). However, this model experiences a significant increase in loss, surpassing 2.0 on DS3 (see Figure 4e).

Conversely, for the balanced datasets, Model 3 shows the most stable and effective performance across all three datasets. While Model 6 also demonstrates strong results with a loss approaching 0.1 on DS1 (see Figure 4b), its performance declines on the balanced DS2 and DS3 datasets, as reflected by higher loss values (see Figures 4d and 4f).

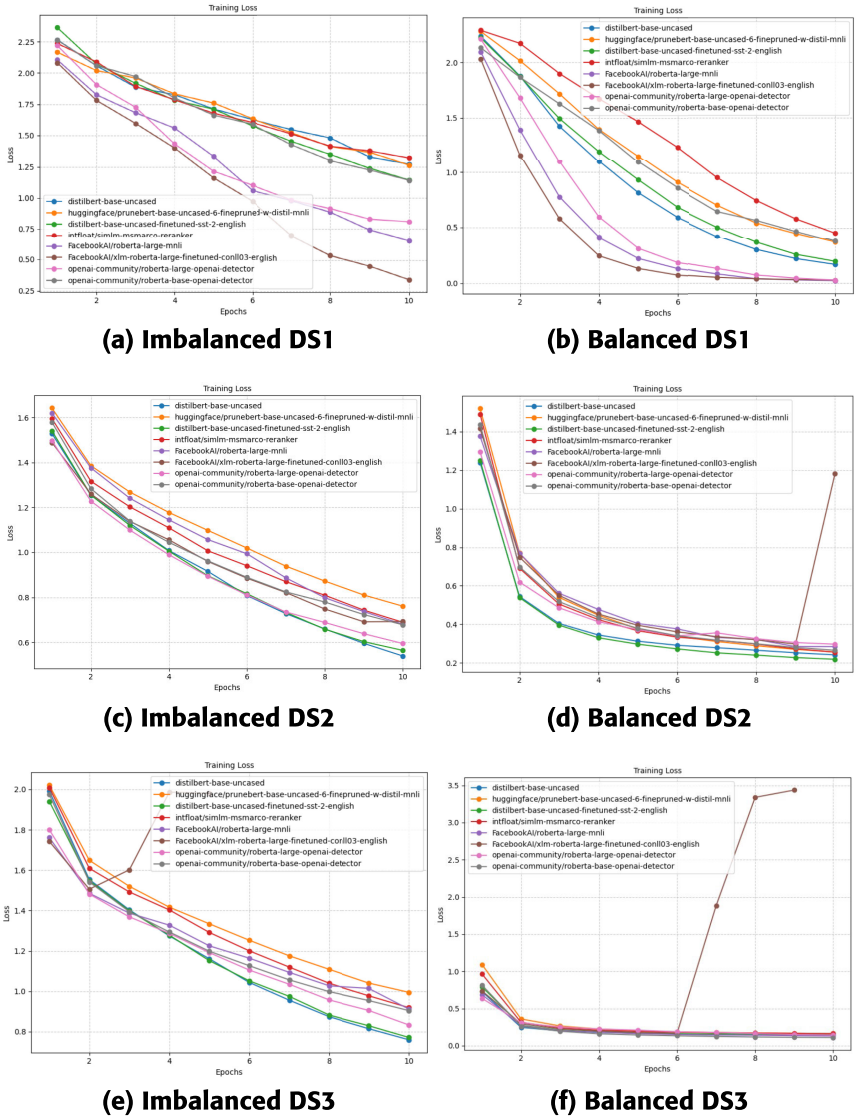


FIGURE 4. Training loss of eight models across three datasets. (a) On Imbalanced SB Curate Data; (b) On Balanced SB Curate Data; (c) On Imbalanced Benchmark Solidity Smart Contract Data; (d) On Balanced Benchmark Solidity Smart Contract Data; (e) On Imbalanced ScrawlD Data; (f) On Balanced ScrawlD Data.

In summary, the training accuracy and loss results demonstrate that our models achieve significantly improved performance on balanced datasets compared to their performance on imbalanced datasets.

B. TESTING EVALUATION

In this section, we focus on visualizing the testing results of our models on the three balanced datasets. For the first dataset (DS1), all eight models effectively detect the 10 types of vulnerabilities, as shown in Figure 5. Model 8 achieves the highest performance (see Figure 6g), while Models 2 and 5 also demonstrate competitive results compared to Model 8 (refer to Figures 5b and 5e).

Similarly, we present the ROC curve results of our models on the second dataset (DS2), as shown in Figure 6. Models 1, 4, 6, and 8 achieve similar performance, each detecting over 90% of the vulnerability types. Meanwhile, Models 2, 3, and 7 are also competitive, with only `TD1` output label showing a detection rate of 89%. In contrast, Model 5 performs the worst in detecting each vulnerability type (see Figure 6e).

Most models perform strongly on the third dataset (DS3), as shown in Table 3.

The AUC-ROC scores for most models range to 1.00 across nearly all vulnerability types, demonstrating their ability to classify vulnerabilities in this dataset accurately. However, Model 5 is an exception, with significantly lower AUC-ROC values in several cases.

- For `ARTHM`, Model 5 achieves only 0.43, whereas other models consistently score above 0.90.
- Similar performance drops can be observed for compound labels like `ARTHM_LE` (0.64 for Model 5 compared to 1.00 for the others) and `ARTHM_LE_RENT` (0.58 for Model 5).

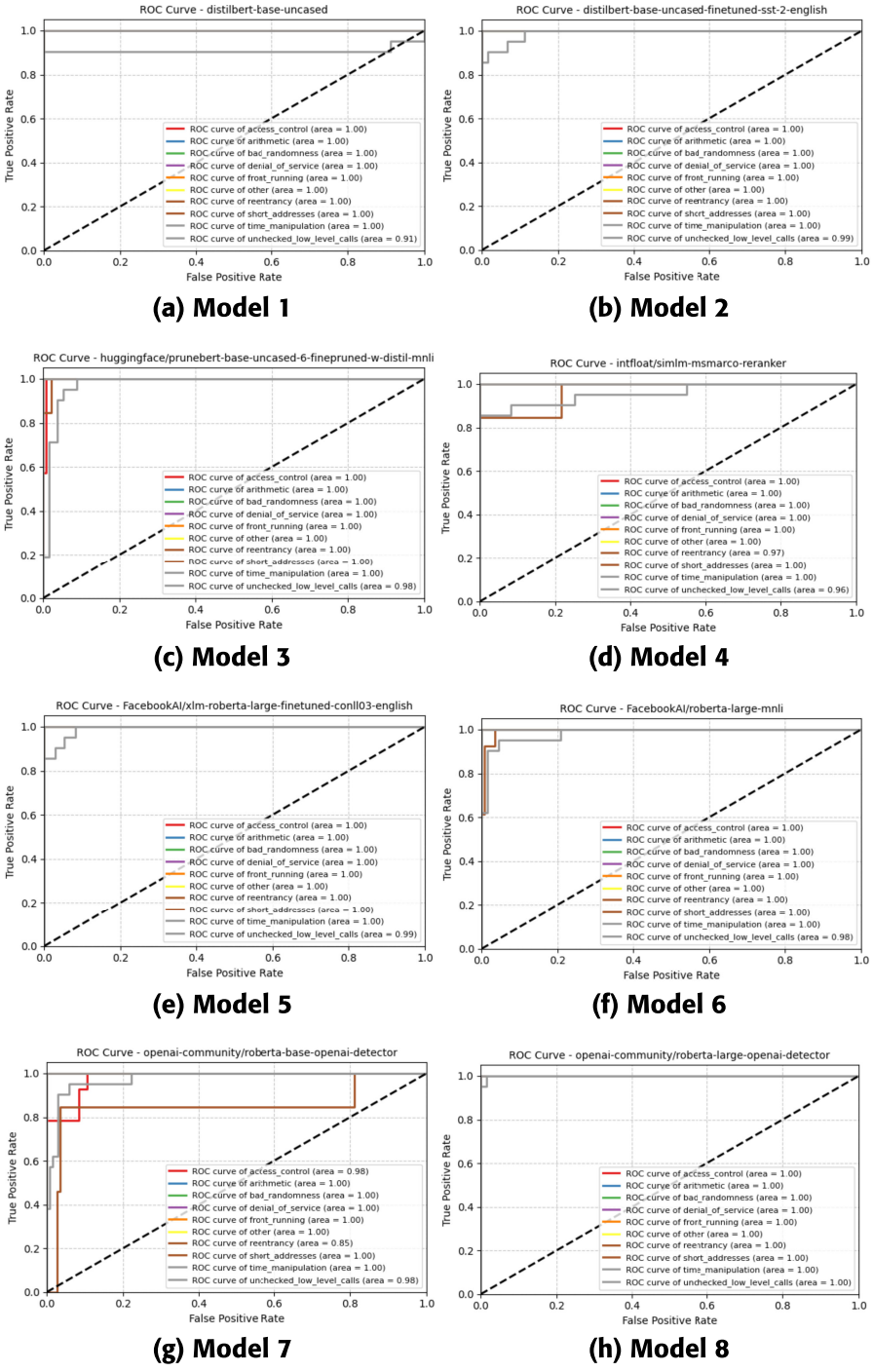


FIGURE 5. ROC curve results for eight models on the sb curate dataset (DS1).
(a) distilbert-base-uncased (Model 1);
(b) huggingface/prunebert-base-uncased-6-finepruned-w-distil-mnli (Model 2);
(c) distilbert-base-uncased-finetuned-sst-2-english (Model 3); (d) intfloat/simlm-msmarco-reranker (Model 4); (e) FacebookAI/roberta-large-mnli (Model 5);
(f) FacebookAI/xlm-roberta-large-finetuned-conll03-english (Model 6);
(g) openai-community/roberta-large-openai-detector (Model 7);
(h) openai-community/roberta-base-openai-detector (Model 8).

- Across most multi-label combinations involving `LE` and `TimeM`, such as `ARTHM_LE_TX-Origin` and `LE_RENT_TimeO`, Model 5's scores are significantly below the near-perfect results of other models.

In contrast, models such as Model 1, Model 2, and Model 3 show consistent AUC-ROC values close to 1.00 across all vulnerability types, including the compound multi-label categories like `ARTHM_LE_TimeM_TimeO` and `LE_TimeM_TimeO`.

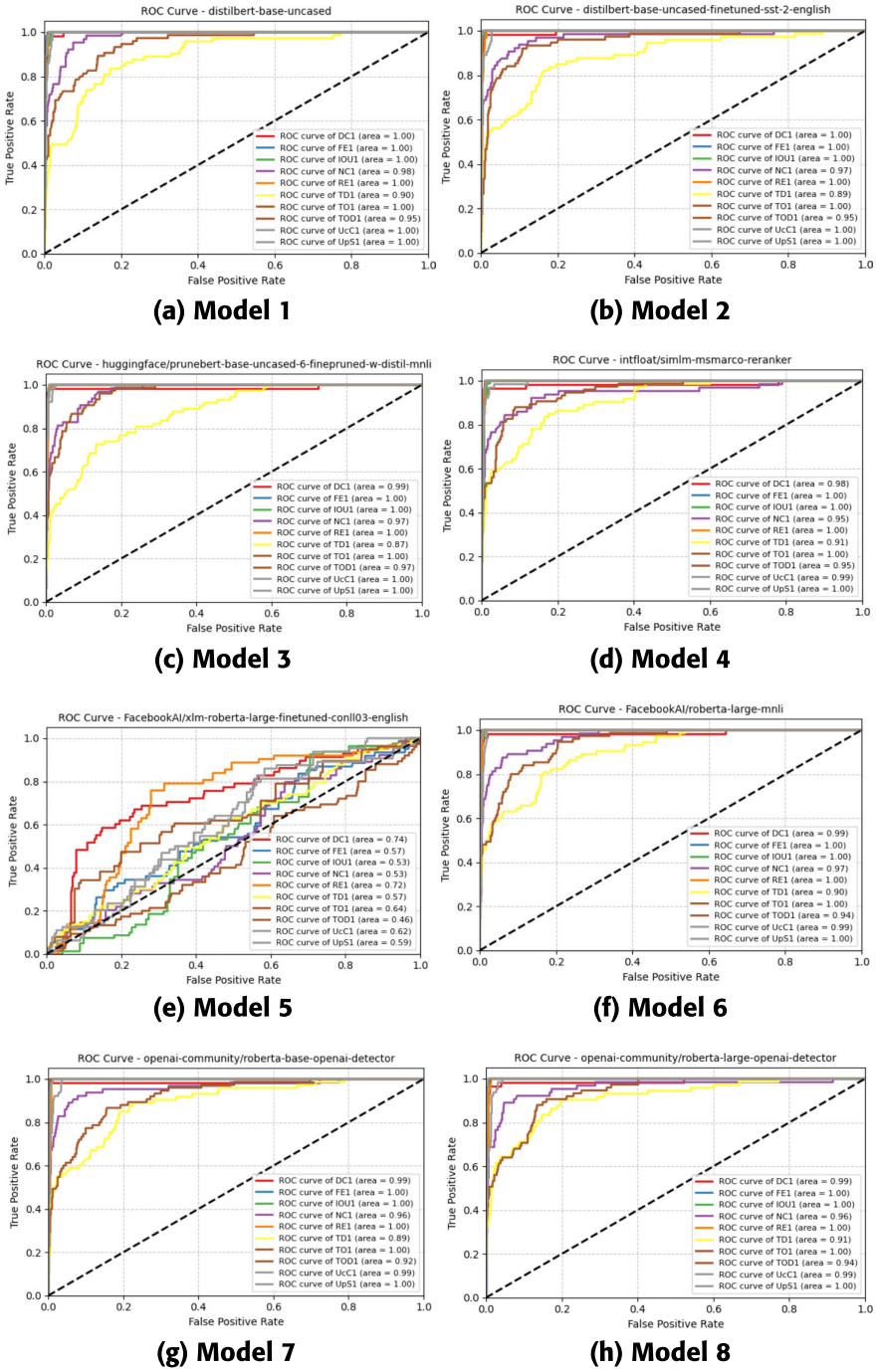


FIGURE 6. **ROC curve results for eight models on the benchmark smart contract dataset (DS2). (a) distilbert-base-uncased (Model 1); (b) huggingface/prunebert-base-uncased-6-finepruned-w-distil-mnli (Model 2); (c) distilbert-base-uncased-finetuned-sst-2-english (Model 3); (d) intfloat/simlm-msmarco-reranker (Model 4); (e) FacebookAI/roberta-large-mnli (Model 5); (f) FacebookAI/xlm-roberta-large-finetuned-conll03-english (Model 6); (g) openai-community/roberta-large-openai-detector (Model 7); (h) openai-community/roberta-base-openai-detector (Model 8).**

Model 8 and Model 7 also show consistently high scores across the board, reinforcing the strength of these models in detecting vulnerabilities across a wide range of categories, including multi-label combinations.

This discrepancy in Model 5's performance suggests that its architecture or fine-tuning might not be well-suited to the multi-label structure and complex patterns of DS3. In contrast, the other models, particularly those leveraging lightweight architectures such as distilBERT (Model 1 and Model 3), demonstrate more effective generalization to multi-label classification tasks.

TABLE 3. AUC-ROC Results for Eight Models on the ScrawID Dataset (DS3). distilbert-base-uncased (Model 1); huggingface/prunebert-base-uncased-6-finepruned-w-distil-mnli (Model 2); distilbert-base-uncased-finetuned-sst-2-english (Model 3); intfloat/simlm-msmarco-reranker (Model 4); FacebookAI/roberta-large-mnli (Model 5); FacebookAI/xlm-roberta-large-finetuned-conll03-english (Model 6); openai-community/roberta-large-openai-detector (Model 7); openai-community/roberta-base-openai-detector (Model 8).

Vulnerability Type	Model 1	Model 2	Model 3	Model 4	Model 5	Model 6	Model 7	Model 8
ARTHM	0.94	0.99	0.98	0.98	0.43	0.97	0.97	0.99
ARTHM_LE	1.00	1.00	1.00	1.00	0.64	1.00	1.00	1.00
ARTHM_LE_RENT	1.00	1.00	1.00	1.00	0.58	1.00	1.00	1.00
ARTHM_LE_RENT_TX-Origins	1.00	1.00	1.00	1.00	0.44	1.00	1.00	1.00
ARTHM_LE_RENT_TimeO	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
ARTHM_LE_TX-Origin	1.00	1.00	1.00	1.00	0.45	1.00	1.00	1.00
ARTHM_LE_TimeM	1.00	1.00	1.00	1.00	0.92	1.00	1.00	1.00
ARTHM_LE_TimeM_TimeO	1.00	1.00	1.00	1.00	0.69	1.00	1.00	1.00
ARTHM_LE_TimeO	1.00	1.00	1.00	1.00	0.49	1.00	1.00	1.00
ARTHM_LE_UE	1.00	1.00	1.00	1.00	0.50	1.00	1.00	1.00
ARTHM_RENT	1.00	1.00	1.00	1.00	0.47	1.00	1.00	1.00
ARTHM_RENT_TX-Origin	1.00	1.00	1.00	1.00	0.86	1.00	1.00	1.00
ARTHM_RENT_TimeM	1.00	1.00	1.00	1.00	0.51	1.00	1.00	1.00
ARTHM_RENT_TimeM_TimeO	1.00	1.00	1.00	1.00	0.78	1.00	1.00	1.00
ARTHM_RENT_TimeO	1.00	1.00	1.00	1.00	0.41	1.00	1.00	1.00
ARTHM_TX-Origin	1.00	1.00	1.00	1.00	0.39	1.00	1.00	1.00
ARTHM_TimeM	1.00	1.00	1.00	1.00	0.52	1.00	1.00	1.00
ARTHM_TimeM_TimeO	1.00	1.00	1.00	1.00	0.51	1.00	1.00	1.00
ARTHM_TimeO	1.00	1.00	1.00	0.99	0.52	1.00	1.00	1.00
ARTHM_UE	1.00	1.00	1.00	1.00	0.22	1.00	1.00	1.00
LE	0.98	0.98	0.98	0.98	0.45	0.99	0.99	1.00
LE_RENT	1.00	1.00	1.00	1.00	0.57	1.00	1.00	1.00
LE_RENT_TimeO	1.00	1.00	1.00	1.00	0.41	1.00	1.00	1.00
LE_TimeM	1.00	1.00	1.00	1.00	0.51	1.00	1.00	1.00
LE_TimeM_TimeO	1.00	1.00	1.00	1.00	0.99	1.00	1.00	1.00
LE_TimeO	1.00	0.99	1.00	1.00	0.51	1.00	1.00	1.00
LE_UE	1.00	1.00	1.00	1.00	0.94	1.00	1.00	1.00
RENT	0.99	0.99	0.99	0.99	0.48	0.99	0.99	1.00
RENT_TimeM	1.00	1.00	1.00	1.00	0.62	1.00	1.00	1.00
RENT_TimeM_TimeO	1.00	1.00	1.00	1.00	0.68	1.00	1.00	1.00
RENT_TimeO	1.00	1.00	1.00	1.00	0.48	1.00	1.00	1.00
TX-Origin_TimeO	1.00	1.00	1.00	1.00	0.91	1.00	1.00	1.00
TimeM	1.00	1.00	1.00	1.00	0.41	1.00	1.00	1.00
TimeM_TimeO	1.00	1.00	1.00	1.00	0.46	1.00	1.00	1.00
TimeO	1.00	1.00	1.00	1.00	0.57	1.00	1.00	1.00
UE	1.00	1.00	1.00	1.00	0.24	1.00	1.00	1.00

TABLE 4. Performance Evaluation Metrics Results for Different Datasets.

Dataset	Model	Precision	Recall	Accuracy	F1
DS 1	Model 5	0.9759	0.9708	0.9708	0.9705
DS 2	Model 2	0.9642	0.9687	0.9687	0.9632
DS 3	Model 8	0.9797	0.9737	0.9737	0.9734

In summary, while most models perform exceptionally well, Model 5 struggles with certain multi-label classifications, indicating that not all transformer-based models perform equally well on diverse vulnerability categories.

C. MULTI-CLASS PERFORMANCE DETECTION WITH OTHER EVALUATION METRIC

1) PERFORMANCE EVALUATION ANALYSIS

We highlight the best-performing models for each dataset in Table 4. This table provides a comprehensive summary of precision, recall, accuracy, and F1 scores, showcasing the effectiveness of the selected models across different datasets.

a: TO DATASET DS1

Model 5 demonstrates outstanding performance with a precision of 0.9759, recall of 0.9708, and F1 score of 0.9705. The high precision indicates that Model 5 is very accurate when predicting a vulnerability type, while the recall shows it effectively identifies most of the actual vulnerabilities. The F1 score, which balances precision and recall, further confirms Model 5's strong overall performance on DS1.

b: TO DATASET DS2

Model 2 leads with a precision of 0.9642 and recall of 0.9687, achieving an F1 score of 0.9632. This model balances precision and recall, making it highly reliable in detecting vulnerabilities. The accuracy of 0.9687 also indicates that Model 2 performs exceptionally well across different classes in this balanced dataset.

c: TO DATASET DS3

Model 8 outperforms others with a precision of 0.9797 and a recall of 0.9737, resulting in an F1 score of 0.9734. This model exhibits the highest precision and recall among the models tested, suggesting it has a superior ability to detect vulnerabilities with minimal false positives and false negatives. The accuracy of 0.9737 reflects Model 8's robust performance across the diverse classes in DS3.

In summary, the performance metrics indicate that the selected models exhibit strong capabilities in vulnerability detection across different datasets. Model 5 excels on DS1, Model 2 is the best performer on DS2, and Model 8 shows

TABLE 5. Performance Metrics for Each Class of DS1.

Class	Precision	Recall	Accuracy	F1 Score
Class 0	0.9333	1.0000	1.0000	0.9655
Class 1	0.9474	1.0000	1.0000	0.9730
Class 2	1.0000	1.0000	1.0000	1.0000
Class 3	1.0000	1.0000	1.0000	1.0000
Class 4	0.9444	1.0000	1.0000	0.9714
Class 5	1.0000	1.0000	1.0000	1.0000
Class 6	0.9462	0.9462	0.9462	0.9462
Class 7	1.0000	1.0000	1.0000	1.0000
Class 8	1.0000	1.0000	1.0000	1.0000
Class 9	0.9889	0.9619	0.9619	0.9505

Note: Class 0 is `access_control`, Class 1 is `arthemetic`, Class 2 is `bad_randomness`, Class 3 is `denial_of_service`, Class 4 is `front_running`, Class 5 is `other`, Class 6 is `reentracy`, Class 7 is `short_address`, Class 8 is `time_manipulation`, Class 9 is `unchecked_low_level_calls`.

TABLE 6. Performance Metrics for Each Class of DS2.

Class	Precision	Recall	Accuracy	F1 Score
Class 0	0.8750	0.9655	0.9655	0.9180
Class 1	0.9385	1.0000	1.0000	0.9683
Class 2	0.9412	0.9877	0.9877	0.9639
Class 3	0.9000	0.9250	0.9250	0.9218
Class 4	0.9841	0.9839	0.9839	0.9813
Class 5	0.9812	0.9438	0.9438	0.9620
Class 6	0.9610	0.9737	0.9737	0.9673
Class 7	0.9586	0.9867	0.9867	0.9617
Class 8	0.9508	0.9675	0.9775	0.9696
Class 9	0.9552	1.0000	1.0000	0.9771

Note: Class 0 is `DC1`, Class 1 is `FE1`, Class 2 is `IOU1`, Class 3 is `NC1`, Class 4 is `NC1`, Class 5 is `RE1`, Class 6 is `TD1`, Class 7 is `TO1`, Class 8 is `TOD1`, Class 9 is `UcC1`, Class 10 is `UpS1`.

the highest performance on DS3. These results highlight the effectiveness of these models in various settings, demonstrating their potential for reliable and accurate vulnerability detection.

2) MULTI-CLASS CLASSIFICATION PERFORMANCE ANALYSIS

For a comprehensive understanding of multi-class classification performance, we present the detailed results of Model 5 for DS1 and DS2 in Tables 5 and 6, respectively. These tables offer an in-depth view of each class's precision, recall, accuracy, and F1 score, reflecting the model's effectiveness in distinguishing between various vulnerability types.

a: TO DATASET DS1

Table 5 provides an exhaustive breakdown of Model 5's performance across different classes. The model achieves perfect precision, recall, and F1 scores for several classes, including Classes 2, 3, 5, 7, and 8, indicating excellent performance in detecting these classes. Class 9 shows slightly lower F1 scores, though still impressive, suggesting that while the model performs very well overall, there is a minor decrease in effectiveness for this class compared to others.

b: TO DATASET DS2

Table 6 presents Model 5's detailed performance metrics for DS2. The model performs exceptionally well across most classes, with Class 1 achieving perfect recall and high precision, making it a standout class. Classes 4 and 9 also show high F1 scores, indicating strong performance.

TABLE 7. Performance Metrics for Each Class of DS3.

Class	Precision	Recall	Accuracy	F1 Score
Class 0	0.9500	0.9786	0.9786	0.9917
Class 1	0.9143	0.9824	0.9824	0.9895
Class 2	0.9130	0.9545	0.9545	0.9333
Class 3	1.0000	1.0000	1.0000	1.0000
Class 4	0.9032	0.9655	0.9655	0.9333
Class 5	1.0000	1.0000	1.0000	1.0000
Class 6	1.0000	1.0000	1.0000	1.0000
Class 7	1.0000	1.0000	1.0000	1.0000
Class 8	0.9524	0.9333	0.9333	0.9889
Class 9	1.0000	1.0000	1.0000	1.0000
Class 10	0.9565	0.9800	0.9800	0.9767
Class 11	1.0000	1.0000	1.0000	1.0000
Class 12	0.9565	0.9462	0.9462	0.9480
Class 13	0.9966	1.0000	1.0000	1.0000
Class 14	1.0000	0.9889	0.9889	0.9412
Class 15	1.0000	1.0000	1.0000	1.0000
Class 16	0.9655	1.0000	1.0000	0.9825
Class 17	1.0000	1.0000	1.0000	1.0000
Class 18	0.9583	0.9200	0.9200	0.9388
Class 19	1.0000	1.0000	1.0000	1.0000
Class 20	0.9647	0.9842	0.9842	0.9822
Class 21	0.9667	0.9355	0.9355	0.9508
Class 22	0.9586	0.9565	0.9565	0.9562
Class 23	0.9286	1.0000	1.0000	0.9630
Class 24	1.0000	1.0000	1.0000	1.0000
Class 25	0.9421	0.9273	0.9273	0.9205
Class 26	1.0000	1.0000	1.0000	1.0000
Class 27	0.9444	0.9391	0.9391	0.9393
Class 28	0.9667	1.0000	1.0000	0.9500
Class 29	0.9048	1.0000	1.0000	0.9500
Class 30	0.9545	0.9545	0.9545	0.9545
Class 31	1.0000	1.0000	1.0000	1.0000
Class 32	0.9615	0.8621	0.8621	0.9091
Class 33	0.9917	1.0000	1.0000	0.9837
Class 34	0.9524	0.9091	0.9091	0.9302
Class 35	1.0000	1.0000	1.0000	1.0000

Note: Class 0 is `ARTHM`, Class 1 is `ARTHM_LE`, Class 2 is `ARTHM_LE_RENT`, Class 3 is `ARTHM_LE_RENT_TX-Orgin`, Class 4 is `ARTHM_LE_RENT_Time`, Class 5 is `ARTHM_LE_TX-Origin`, Class 6 is `ARTHM_LE_TimeM`, Class 7 is `ARTHM_LE_TimeM_TimeO`, Class 8 is `ARTHM_LE_TimeO`, Class 9 is `ARTHM_LE_UE`, Class 10 is `ARTHM_RENT`, Class 11 is `ARTHM_RENT_TX-Origin`, Class 12 is `ARTHM_RENT_TimeM`, Class 13 is `ARTHM_RENT_TimeM_TimeO`, Class 14 is `ARTHM_RENT_TimeO`, Class 15 is `ARTHM_TX-Origin`, Class 16 is `ARTHM_TimeM`, Class 17 is `ARTHM_TimeM_TimeO`, Class 18 is `ARTHM_TimeO`, Class 19 is `ARTHM_UE`, Class 20 is `LE`, Class 21 is `LE_RENT`, Class 22 is `LE_RENT_TimeO`, Class 23 is `LE`, Class 24 is `LE_TimeM_TimeO`, Class 25 is `LE_TimeO`, Class 26 is `LE_UE`, Class 27 is `RENT`, Class 28 is `RENT_TimeM`, Class 29 is `RENT_TimeM_TimeO`, Class 30 is `RENT_TimeO`, Class 31 `Tx-Origin_TimeO`, Class 32 is `TimeM`, Class 33 is `TimeM_TimeO`, Class 34 is `TimeO`, Class 35 is `UE`.

However, Class 3 has slightly lower F1 and accuracy scores than other classes, suggesting there may be room for improvement in detecting this particular class.

These tables illustrate that Model 5 exhibits high multi-class classification performance for DS1 and DS2. The precision, recall, and F1 scores reflect the model's robustness in distinguishing between different vulnerability types, though some classes show slightly varied performance levels. This detailed analysis underscores the model's effectiveness and highlights areas for potential enhancement in specific classes.

D. LARGE NUMBER OF MULTI-CLASS DETECTION PERFORMANCE ANALYSIS

The large number of multi-class detection performances of our models on DS3 is detailed in Table 7. This table provides

precision, recall, accuracy, and F1 scores for each class, offering a comprehensive view of the model's effectiveness across a broad range of vulnerability labels.

a: TO DATASET DS3

Table 7 illustrates the large number of multi-class detection performance of our models across 36 classes. The model exhibits exceptional performance across the majority of classes:

- High Performers: Classes 3, 5, 6, 7, 9, 11, 13, 15, 17, 19, 24, 26, 31, and 35 all demonstrate perfect precision, recall, and F1 scores of 1.0000, indicating flawless detection and classification of these classes. This suggests the model is highly effective at identifying these labels without missing or incorrectly classifying them.
- Consistently Strong Performance: Many other classes also show high performance, precision, recall, and F1 scores above 0.90. For example, Class 10 (precision: 0.9565, recall: 0.9800, F1 score: 0.9767) and Class 20 (precision: 0.9647, recall: 0.9842, F1 score: 0.9822) exhibit particularly robust metrics.
- Areas for Improvement: Some classes, such as Class 18 (precision: 0.9583, recall: 0.9200, F1 score: 0.9388) and Class 32 (precision: 0.9615, recall: 0.8621, F1 score: 0.9091), show slightly lower scores. While these metrics are still strong, they indicate potential areas where the model's performance could be further optimized.

The model's ability to handle a broad set of multi-class vulnerabilities demonstrates its versatility and robustness, with high performance across various classes. The detailed performance metrics underscore the model's effectiveness in multi-class detection tasks and highlight areas for potential refinement.

V. DISCUSSION

A. SUMMARY OF EXPERIMENTAL RESULTS

Our experiments evaluated multiple models across three datasets (DS1, DS2, and DS3) to assess their performance in multi-class and multi-class vulnerability detection.

- Dataset DS1: Model 5 achieved the highest scores across all metrics with a precision of 0.9759, recall of 0.9708, accuracy of 0.9708, and F1 score of 0.9705. The model demonstrated excellent performance in classifying various vulnerability types, with particularly strong results for Classes 2, 3, 5, 7, and 8.
- Dataset DS2: Model 2 emerged as the top performer with a precision of 0.9642, recall of 0.9687, accuracy of 0.9687, and an F1 score of 0.9632. The model performed consistently well across all classes, with notable accuracy in detecting vulnerabilities in Classes 1, 4, 6, 7, and 9.
- Dataset DS3: Model 8 showed superior performance with a precision of 0.9797, recall of 0.9737, accuracy of 0.9737, and an F1 score of 0.9734. This model excelled in a multi-class context, especially in Classes 3, 5, 6, 7, 11, 13, 15, and 19, achieving perfect or near-perfect scores.

B. ADVANTAGE AND DISADVANTAGE OF THE PROPOSED METHOD

The advantages of our method are as follows:

- High Overall Performance: The models demonstrated exceptional performance metrics across all datasets, indicating their effectiveness in accurately detecting and classifying vulnerabilities. High precision and recall scores suggest that the models are reliable and effective in identifying vulnerabilities with minimal false positives and negatives.
- Robust Multi-Class Capability: The models exhibited strong performance in multi-class scenarios, showcasing their versatility and ability to handle complex, diversity, and large multi-class classifications. This is particularly evident from the detailed class-wise metrics provided for each dataset.
- Consistent Results Across Datasets: High-performance consistency across different datasets highlights the models' generalizability. This indicates that the models can effectively handle various vulnerabilities and are not overly specific to a single dataset.

The disadvantages are:

- Performance Variability: Despite strong overall performance, some classes showed variability in metrics. For instance, Class 6 in DS1 and Class 18 in DS3 had lower recall or F1 scores. This variability suggests that the models may struggle with certain classes due to insufficient representation or inherent difficulty.
- Risk of Overfitting: The high performance on the provided datasets raises concerns about overfitting. There is a risk that the models are tailored too closely to the datasets, which might limit their effectiveness on new or unseen data.
- Computational and Resource Demands: The models' complexity and the computational resources required for training and inference can be significant. This might limit their applicability in environments with limited resources or where real-time performance is critical.

C. FUTURE WORKS

To address the limitations identified and to enhance the models' performance, particularly in the context of multi-label detection in Solidity code, we propose the following directions for future research:

- Deep Multi-Label Detection: Extend the models to address deep multi-label detection tasks where each label encompasses multiple attributes beyond simple categorization. In the context of Solidity code, this involves recognizing the presence of vulnerabilities and extracting detailed contextual information such as line number, function, severity, and specific vulnerability types.
- Enhanced Label Representation: Develop methods to handle complex multi-label scenarios where each

label represents a series of interconnected values. For Solidity code, this means capturing detailed aspects of vulnerabilities, such as the precise line of code, the function where the vulnerability occurs, and the severity of the issue. Incorporate techniques that allow for hierarchical and structured label representations to better model these intricate relationships.

- Optimizing for Computational Efficiency: Investigate approaches to optimize the models for computational efficiency, including model compression techniques like pruning and quantization. This will help reduce the computational resources required, making the models more feasible for real-time and large-scale applications.

VI. CONCLUSION

This study investigated the effectiveness of fine-tuning pre-trained transformer encoder models for vulnerability detection in smart contracts. By leveraging eight distinct pre-trained LLMs, we evaluated their performance on three different datasets: SB Curate (DS1), Benmark Solidity Smart Contract (DS2), and ScrawID (DS3). Our evaluation encompassed imbalanced and balanced dataset scenarios to assess the impact of data distribution on model performance.

Key findings from our experiments include:

- **Impact of Dataset Balancing:** Balancing the datasets significantly improved model performance. Models trained on balanced datasets exhibited higher accuracy, lower loss, and better ROC curve metrics than those trained on imbalanced datasets. This improvement highlights the critical role of data preprocessing in enhancing model performance, particularly for detecting minority class vulnerabilities.
- **Model Performance Variability:** The models demonstrated varying levels of effectiveness across the different datasets and training scenarios. The fine-tuning process enabled the LLMs to adapt their pre-trained knowledge to the specific task of vulnerability detection, showcasing their versatility in handling diverse data inputs and vulnerability types.
- **Evaluation Metrics:** The analysis of accuracy, loss, and ROC curves provided a comprehensive view of model performance. Models trained on balanced datasets achieved better accuracy and lower loss and displayed improved ROC curves, indicating more reliable and robust performance in detecting vulnerabilities.

Our findings underscore the importance of dataset quality and balance in training machine learning models for vulnerability detection. Future work could explore additional strategies for handling imbalanced data, such as advanced sampling techniques or synthetic data generation, to further enhance model performance. Additionally, investigating the impact of various model architectures and hyperparameter settings could provide deeper insights into optimizing LLMs for specific security tasks.

Overall, this study contributes valuable insights into the effective use of pre-trained transformer models for software security applications, paving the way for more robust and reliable vulnerability detection systems in smart contracts and beyond.

REFERENCES

[1] *Security Analysis Tool for EVM Bytecode*. Accessed: Jun. 20, 2024. [Online]. Available: https://github.com/ConsenSys/mythril
[2] *Static Analyzer for Solidity and Vyper*. Accessed: Jun. 20, 2024. [Online]. Available: https://github.com/crytic/slither
[3] *Ethereum Smart Contract Fuzzer*. Accessed: Jun. 20, 2024. [Online]. Available: https://github.com/crytic/echidna
[4] *Symbolic Execution Tool*. Accessed: Jun. 20, 2024. [Online]. Available: https://github.com/trailofbits/manticore
[5] Z. Liu, P. Qian, X. Wang, Y. Zhuang, L. Qiu, and X. Wang, "Combining graph neural networks with expert knowledge for smart contract vulnerability detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 2, pp. 1296–1310, Feb. 2023.
[6] Y. Liu, F. R. Yu, X. Li, H. Ji, and V. C. M. Leung, "Blockchain and machine learning for communications and networking systems," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 2, pp. 1392–1431, 2nd Quart., 2020.
[7] E. Shi, Y. Wang, H. Zhang, L. Du, S. Han, D. Zhang, and H. Sun, "Towards efficient fine-tuning of pre-trained code models: An experimental study and beyond," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2023, pp. 39–51.
[8] K. Napier, T. Bhowmik, and S. Wang, "An empirical study of text-based machine learning models for vulnerability detection," *Empirical Softw. Eng.*, vol. 28, no. 2, pp. 1–26, Mar. 2023.
[9] Q. Zhou, K. Zheng, K. Zhang, L. Hou, and X. Wang, "Vulnerability analysis of smart contract for blockchain-based IoT applications: A machine learning approach," *IEEE Internet Things J.*, vol. 9, no. 24, pp. 24695–24707, Dec. 2022.
[10] M. Al-Essa and A. Appice, "Dealing with imbalanced data in multi-class network intrusion detection systems using xgboost," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discovery Databases*, 2021, pp. 5–21.
[11] T.-T.-H. Le, Y. E. Oktian, and H. Kim, "XGBoost for imbalanced multiclass classification-based industrial Internet of Things intrusion detection systems," *Sustainability*, vol. 14, no. 14, p. 8707, Jul. 2022.
[12] T.-T.-H. Le, Y. Shin, M. Kim, and H. Kim, "Towards unbalanced multiclass intrusion detection with hybrid sampling methods and ensemble classification," *Appl. Soft Comput.*, vol. 157, May 2024, Art. no. 111517.
[13] F. He, F. Li, and P. Liang, "Enhancing smart contract security: Leveraging pre-trained language models for advanced vulnerability detection," *IET Blockchain*, vol. 2, pp. 2–10, Mar. 2024.
[14] J. Kim, T.-T.-H. Le, S. Lee, and H. Kim, "Leveraging fine-tuning DistilBERT for detecting vulnerabilities in Ethereum smart contracts," in *Proc. IEEE Int. Conf. Platform Technol. Service*, Jeju-si, South Korea, Aug. 2024. [Online]. Available: https://bit.ly/p24proc
[15] C. Thapa, S. I. Jang, M. E. Ahmed, S. Camtepe, J. Pieprzyk, and S. Nepal, "Transformer-based language models for software vulnerability detection," in *Proc. 38th Annu. Comput. Secur. Appl. Conf.*, Dec. 2022, pp. 481–496.
[16] B. Boi, C. Esposito, and S. Lee, "Smart contract vulnerability detection: The role of large language model (LLM)," *ACM SIGAPP Appl. Comput. Rev.*, vol. 24, no. 2, pp. 19–29, Jun. 2024.
[17] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, "GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, Apr. 2024, pp. 1–13.
[18] B. Boi, C. Esposito, and S. Lee, "VulnHunt-GPT: A smart contract vulnerabilities detector based on OpenAI ChatGPT," in *Proc. 39th ACM/SIGAPP Symp. Appl. Comput.*, Apr. 2024, pp. 1517–1524.
[19] M. Fu and C. Tantithamthavorn, "LineVul: A transformer-based line-level vulnerability prediction," in *Proc. IEEE/ACM 19th Int. Conf. Mining Softw. Repositories (MSR)*, May 2022, pp. 608–620.
[20] Y. Chen, Z. Sun, Z. Gong, and D. Hao, "Improving smart contract security with contrastive learning-based vulnerability detection," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, Apr. 2024, pp. 1–11.
[21] C. Jiang, G. Sun, J. Shen, B. Yue, and Y. Zhang, "Multi-label detection method for smart contract vulnerabilities based on expert knowledge and pre-training technology," in *Proc. Int. Conf. Algorithms Archit. Parallel Process.*, 2023, pp. 299–312.

[22] S.-J. Hwang, S.-H. Choi, J. Shin, and Y.-H. Choi, "CodeNet: Code-targeted convolutional neural network architecture for smart contract vulnerability detection," *IEEE Access*, vol. 10, pp. 32595–32607, 2022.
[23] A. Ghaleb, "Towards effective static analysis approaches for security vulnerabilities in smart contracts," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, Oct. 2022, pp. 1–5.
[24] X. Yu, H. Zhao, B. Hou, Z. Ying, and B. Wu, "DeeSCVHunter: A deep learning-based framework for smart contract vulnerability detection," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2021, pp. 1–8.
[25] F. Mi, Z. Wang, C. Zhao, J. Guo, F. Ahmed, and L. Khan, "VSCL: Automating vulnerability detection in smart contracts with deep learning," in *Proc. IEEE Int. Conf. Blockchain Cryptocurrency (ICBC)*, May 2021, pp. 1–9.
[26] M. Ciniselli, N. Cooper, L. Pascarella, A. Mastropaolo, E. Aghajani, D. Poshyvanyk, M. Di Penta, and G. Bavota, "An empirical study on the usage of transformer models for code completion," *IEEE Trans. Softw. Eng.*, vol. 48, no. 12, pp. 4818–4837, Dec. 2022.
[27] Z. Chu, Y. Wan, Q. Li, Y. Wu, H. Zhang, Y. Sui, G. Xu, and H. Jin, "Graph neural networks for vulnerability detection: A counterfactual explanation," 2024, *arXiv:2404.15687*.
[28] J. Cheng, Y. Chen, Y. Cao, and H. Wang, "A vulnerability detection framework by focusing on critical execution paths," *Inf. Softw. Technol.*, vol. 174, Oct. 2024, Art. no. 107517.
[29] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
[30] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," OpenAI, 2018. [Online]. Available: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf and https://openai.com/index/language-unsupervised/
[31] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," 2019, *arXiv:1910.01108*.
[32] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.
[33] M. Soud, W. Nuutinen, and G. Liebel, "Soley: Identification and automated detection of logic vulnerabilities in Ethereum smart contracts using large language models," 2024, *arXiv:2406.16244*.
[34] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, pp. 1–9, 2019.
[35] T. B. Brown, "Language models are few-shot learners," 2020, *arXiv:2005.14165*.
[36] M. Sathvik and H. Mazumdar, "Detection of malicious smart contracts by fine-tuning GPT-3," *Secur. Privacy*, vol. 2, p. e430, Jun. 2024.
[37] *SB Curated Dataset*. Accessed: Jun. 20, 2024. [Online]. Available: https://github.com/smartbugs/smartbugs-curated
[38] *A Benchmark Dataset of Solidity Smart Contracts*. Accessed: Jun. 20, 2024. [Online]. Available: https://zenodo.org/records/7606610
[39] *ScrawlD Dataset*. Accessed: Jun. 20, 2024. [Online]. Available: https://github.com/sujeetc/ScrawlD
[40] Z. Xu, J. Wang, C. Zha, X. Li, and H. Yin, "SmartLLM: A new Oracle system for smart contracts calling large language models," in *Proc. IEEE 22nd Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, vol. 11, Nov. 2023, pp. 2668–2675.
[41] S. Zeng, R. Chen, H. Zhang, and J. Wang, "A high-performance smart contract vulnerability detection scheme based on BERT," in *Proc. IEEE 29th Int. Conf. Parallel Distrib. Syst. (ICPADS)*, vol. 18, Dec. 2023, pp. 653–658.
[42] A. Z. H. Yang, H. Tian, H. Ye, R. Martins, and C. Le Goues, "Security vulnerability detection with multitask self-instructed fine-tuning of large language models," 2024, *arXiv:2406.05892*.
[43] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?" *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3280–3296, Sep. 2022.
[44] *WordPiece Tokenizer*. Accessed: Jun. 20, 2024. [Online]. Available: https://huggingface.co/learn/nlp-course/chapter6/6?fw=pt
[45] *Model 1: distilbert-base-uncased*. Accessed: May 15, 2024. [Online]. Available: https://huggingface.co/distilbert/distilbert-base-cased
[46] *Model 2: prunebert-base-uncased-6-finepruned-w-distil-mnli*. Accessed: May 15, 2024. [Online]. Available: https://huggingface.co/huggingface/prunebert-base-uncased-6-finepruned-w-distil-mnli
[47] *Model 3: distilbert-base-uncased-finetuned-sst-2-english*. Accessed: May 15, 2024. [Online]. Available: https://huggingface.co/distilbert/distilbert-base-uncased-finetuned-sst-2-english
[48] *Model 4: simlm-msmarco-reranker*. Accessed: May 15, 2024. [Online]. Available: https://huggingface.co/intfloat/simlm-msmarco-reranker
[49] *Model 5: roberta-large-mnli*. Accessed: May 15, 2024. [Online]. Available: https://huggingface.co/FacebookAI/roberta-large-mnli
[50] *Model 6: xlm-roberta-large-finetuned-conll03-english*. Accessed: May 15, 2024. [Online]. Available: https://huggingface.co/FacebookAI/xlm-roberta-large-finetuned-conll03-english
[51] *Model 7: roberta-large-openai-detector*. Accessed: May 15, 2024. [Online]. Available: https://huggingface.co/openai-community/roberta-large-openai-detector
[52] *Model 8: openai-community/roberta-base-openai-detector*. Accessed: May 15, 2024. [Online]. Available: https://huggingface.co/openai-community/roberta-base-openai-detector



THI-THU-HUONG LE received the bachelor's degree from Hung Yen University of Technology and Education (HYUTE), Vietnam, in 2007, the master's degree from Hanoi University of Science and Technology (HUST), in 2013, and the Ph.D. degree from Pusan National University (PNU), South Korea, in 2020. She is a Research Professor with the Blockchain Platform Research Center, PNU. She has three years of experience as a Postdoctoral Researcher with PNU, in 2020. She had seven years of experience as a Lecturer with HYUTE. She has participated in machine learning projects, such as NILM, IDS, AI industry 4.0, AI security, and deep learning-based CFD. Her research interests include machine learning, deep learning, generative AI, data analysis, explainable AI, and signal processing. She has served several academic services, such as a guest editor, a technical committee, and a reviewer.



JAEHYUN KIM received the bachelor's degree in computer engineering from Pusan National University, South Korea, in 2023, where he is currently pursuing the master's degree. His current research interests include blockchain and the convergence of blockchain with artificial intelligence (AI), focusing on how these technologies can be integrated to enhance security, transparency, and efficiency across various domains.



SANGMYEONG LEE received the bachelor's degree in computer engineering from Pusan National University, South Korea, in 2024, where he is currently pursuing the master's degree in computer science. His current research interests include blockchain and security.



HOWON KIM (Member, IEEE) received the bachelor's degree from Kyungpook National University (KNU), and the Ph.D. degree from Pohang University of Science and Technology (POSTECH). He is currently a Professor with the Department of Electrical and Computer Engineering, the Chief of the Energy Internet of Things (IoT), IT Research Center (ITRC), and the Chief of the Information Security Education Center (ISEC), Pusan National University (PNU). Before joining PNU, he was with the Electronics and Telecommunications Research Institute (ETRI) as a Team Leader for ten years, in December 1998. He was a Postdoctoral Researcher, he visited the Chair of Communication Security Group (COSY), Ruhr-University Bochum, Germany, from July 2003 to June 2004.